

Technická zpráva

Boston Bruins Fan Shop

PHP 8.3 + SQLite e-shop

Projekt: PRG — 2. fáze (převod statického frontendu na dynamický backend)

Autor: Matyáš Géryk

Datum: 2026

Úvod

Tato technická zpráva dokumentuje projekt Boston Bruins Fan Shop — plně funkční e-shop napsaný v jazyce PHP 8.3 s databází SQLite. Projekt vznikl jako druhá fáze semestrální práce z předmětu PRG, jejímž cílem bylo převést statický frontend (HTML/CSS) na dynamickou aplikaci s databází, košíkem, vícekrokovým checkoutem a uložením objednávek.

Aplikace simuluje oficiální fan shop hokejového týmu Boston Bruins z NHL pro české a slovenské fanoušky. Obsahuje katalog produktů rozdělený do kategorií, detail produktu s variantami (velikost / barva), nákupní košík uložený v session, třístupňový proces objednávky se server-side validací, vyhledávání, kontaktní formulář a statické informační stránky.

Cíle projektu

- Převést statický HTML/CSS frontend z 1. fáze na plně dynamickou PHP aplikaci.
- Implementovat relační datový model v SQLite (kategorie, produkty, varianty, obrázky, objednávky).
- Vytvořit funkční nákupní košík s podporou variant produktů.
- Postavit vícekrokový checkout s validací a perzistencí dat v session (PRG vzor).
- Aplikovat základní principy bezpečnosti — prepared statements, escapování výstupu, serverside validace.
- Dodržet čistou architekturu (Repository + DTO pattern, oddělení logiky od pohledu).

Použité technologie

- PHP 8.3 s direktivou declare(strict_types=1) ve všech souborech.
- SQLite 3 přes PDO s prepared statements.
- PHP Sessions pro košík a stav více krokové objednávky.
- Architektonické vzory: Repository, DTO, Singleton (Database), PRG (Post/Redirect/Get).
- Frontend: ručně psané HTML5 + CSS3 (bez frameworku), responzivní design.
- Vestavěný PHP server (php -S) s vlastním router.php.

Struktura projektu

Projekt používá plochou strukturu vstupních PHP skriptů v kořeni (každá URL odpovídá jednomu souboru, např. /kosik.php) a oddělenou aplikační vrstvu v adresáři src/. Tento přístup je v souladu s klasickou LAMP konvencí a usnadňuje nasazení na sdílený hosting.

Adresářová struktura

```
/
├── index.php, kategorie.php, produkty.php, produkt.php
├── kosik.php
├── objednavka-1.php, objednavka-2.php, objednavka-3.php
├── objednavka-potvrzeni.php
├── vyhledavani.php, o-nas.php, kontakt.php, 404.php
├── router.php — router pro vestavěný PHP server
├── src/
│   ├── bootstrap.php — autoload, session, helpery e(), price(), redirect()
│   ├── Database.php — PDO singleton
│   ├── Cart.php — košík v session
│   ├── Validator.php — fluent validátor formulářů
│   ├── DTO/ — datové objekty (ProductDTO, CategoryDTO, ...)
│   └── Repository/ — přístup k DB (ProductRepository, OrderRepository, ...)
├── partials/ — header.php, footer.php, product-card.php
├── database/
│   ├── init.php — vytvoří databázi a naplní ji vzorovými daty
│   └── eshop.db — SQLite soubor (negitujeme)
└── assets/css/main.css
```

Architektonické vrstvy

Aplikace je rozdělena do tří jasně oddělených vrstev:

- Prezenční vrstva — soubory v kořeni a partials. Zpracují vstup, zavolají repository a vykreslí HTML.
- Doménová vrstva — DTO třídy v src/DTO/. Nesou data mezi DB a šablonou typově bezpečně.
- Datová vrstva — repository třídy v src/Repository/. Obalují SQL dotazy a vrací DTO.

Singleton Database drží jedno PDO připojení po dobu requestu. Třída Cart je statická obálka nad `$_SESSION`, která drží položky v košíku jako asociativní pole klíčované kombinací `product_id` + `variant_id`.

Routování

Pro lokální vývoj se používá vestavěný PHP server s vlastním `router.php`. Router servíruje statické soubory přímo (CSS, obrázky), kořen směřuje na `index.php`, ostatní URL mapuje na odpovídající `.php` soubor a v případě neexistence vrací vlastní stránku `404.php` s HTTP statusem 404.

Popis funkcí

Katalog a detail produktu

Hlavní stránka (`index.php`) zobrazuje hero banner, doporučené produkty (metoda `ProductRepository::getFeatured(4)`) a přehled kategorií s počty produktů. Stránka `kategorie.php` vypisuje všechny kategorie, `produkty.php?slug=...` zobrazí produkty dané kategorie a `produkt.php?slug=...` ukáže detail s galerií, variantami a popisem. Neexistující slug vede přes `require '404.php'` na standardní 404.

Košík

Košík je řešen třídou `Cart`, která pracuje s `$_SESSION['cart']`. Podporuje tři POST akce — `add`, `update` a `remove`. Po každé akci dochází k PRG redirectu na `/kosik.php`, takže obnovení stránky znovu neodešle formulář. Položky s různou variantou stejného produktu jsou v košíku samostatné záznamy.

Třístupňový checkout

- `objednavka-1.php` — formulář s dodací adresou (jméno, příjmení, e-mail, telefon, ulice, město, PSČ, poznámka). Validuje se přes třídu `Validator` (`required`, `minLength/maxLength`, `email`, `regex` na telefon a PSČ).
- `objednavka-2.php` — výběr způsobu dopravy a platby z dat v DB. Validace zahrnuje kontrolu, že vybraná hodnota je v povoleném číselníku (validátor `in`).
- `objednavka-3.php` — souhrn objednávky, souhlas s obchodními podmínkami, uložení zákazníka a objednávky do DB v rámci jedné transakce, vyčištění košíku a redirect na `objednavka-potvrzeni.php`.

Mezi kroky se data drží v `$_SESSION['order']`. Každý krok kontroluje předpoklady (např. neprázdný košík, vyplněná adresa) a v případě chyby přesměruje uživatele zpět.

Vyhledávání a ostatní stránky

- `vyhledavani.php?q=...` — fulltext nad názvem a popisem produktu (`LIKE %q%` v repository).
- `kontakt.php` — formulář s validací a flash zprávou v session po úspěšném odeslání (PRG).

- o-nas.php — statická informační stránka.
- 404.php — vlastní stránka pro neexistující obsah, vrací HTTP 404.

Schéma databáze

Databáze běží v SQLite (soubor database/eshop.db) a inicializuje se skriptem database/init.php, který zároveň naplní tabulky vzorovými daty. Model je normalizovaný do třetí normální formy.

Přehled tabulek

Tabulka	Účel	Klíčové sloupce
categories	Kategorie produktů	id, slug, name, description
products	Produkty	id, category_id, slug, name, short_description, description, price, sale_price, is_featured
product_images	Obrázky produktu (1:N)	id, product_id, path, alt, is_main, sort_order
product_variants	Varianty (velikost / barva)	id, product_id, name, value, price_modifier
shipping_methods	Číselník dopravy	id, code, name, price
payment_methods	Číselník plateb	id, code, name, fee
customers	Zákazníci	id, first_name, last_name, email, phone, street, city, zip, note
orders	Objednávky	id, customer_id, order_number, shipping_id, payment_id, items_total, shipping_price, payment_fee, total, created_at
order_items	Položky objednávky (1:N)	id, order_id, product_id, variant_id, name, variant_name, quantity, unit_price, subtotal

Vztahy mezi tabulkami

- categories 1 — N products (každý produkt patří do jedné kategorie).
- products 1 — N product_images (galerie obrázků, jeden je označen is_main).
- products 1 — N product_variants (velikost, barva s případnou cenovou úpravou).
- customers 1 — N orders (zákazník je vytvořen při dokončení objednávky).

- orders 1 — N order_items (snapshot položek v okamžiku objednávky — název i cena se kopírují, aby pozdější změna produktu neovlivnila historii).
- shipping_methods a payment_methods jsou číselníky referencované z tabulky orders.

ER schéma (textově)

```
categories (1) —< products (1) —< product_images
products (1) —< product_variants customers (1) —< orders
(1) —< order_items shipping_methods (1) —< orders
payment_methods (1) —< orders
```

Všechny cizí klíče jsou deklarovány s ON DELETE CASCADE tam, kde má smysl

(obrázky a varianty při smazání produktu, položky při smazání objednávky). Cizí klíče v SQLite jsou aktivovány direktivou PRAGMA foreign_keys = ON v Database.php.

Návod na spuštění

Požadavky

- PHP 8.3 nebo novější.
- PHP rozšíření pdo_sqlite (standardně součást distribuce).
- Příkazová řádka (bash / PowerShell).
- Git (pouze pokud chcete projekt klonovat z GitHubu).

Lokální instalace

```
# 1) Naklonujte repozitář git clone
https://github.com/<uzivatel>/bostonbruins-shop.git cd boston-bruins-shop

# 2) Inicializujte databázi a naplňte ji vzorovými daty php
database/init.php

# 3) Spusťte vestavěný PHP server s vlastním routerem php -S
localhost:8000 router.php
```

Aplikace bude dostupná na <http://localhost:8000>.

Spuštění v GitHub Codespaces

```
php database/init.php php -S
0.0.0.0:8000 router.php
```

Codespaces automaticky nabídne port forwarding na portu 8000 a otevře aplikaci v prohlížeči.

Reset databáze

Pro vyčištění a znovunaplnění dat stačí smazat soubor database/eshop.db a znovu spustit php database/init.php.

Testovací scénář

- Otevřít hlavní stránku → ověřit zobrazení doporučených produktů a kategorií.
- Vstoupit do kategorie, otevřít detail produktu, zvolit variantu a přidat do košíku.
- Otevřít košík, změnit množství a odebrat položku.
- Projít třístupňový checkout, ověřit validaci (vyplnit nevalidní e-mail, prázdné PSČ).
- Dokončit objednávku a ověřit záznam v tabulkách customers, orders, order_items.
- Vyhledat produkt podle názvu a ověřit fungování fulltextu.
- Otevřít neexistující URL a ověřit vlastní stránku 404.

Bezpečnost a závěr

Aplikace dodržuje základní bezpečnostní principy: všechny SQL dotazy jdou přes prepared statements (PDO), veškerý uživatelský výstup je escapován helperem e() nad htmlspecialchars() (prevence XSS) a všechny formuláře validujeme na serveru bez ohledu na klientské kontroly. Cena za dopravu a platbu se počítá vždy na serveru z DB hodnot, takže ji nelze podvrhnout přes POST.

Nejtěžší na celém projektu bylo rozřezání původního statického HTML kódu do PHP partials (hlavička, patička) a správné napojení dynamických dat z databáze tak, aby se nerozbil design.

Co bych doplnil: Kdybych měl víc času, přidal bych přihlašování uživatelů nebo slevové kódy.

Zhodnocení projektu: Nejtěžší na celém projektu bylo správně propojit nákupní košík v session s PRG vzorem, aby se zamezilo dvojitému odesílání formulářů. Zpětně bych si od začátku mnohem lépe strukturoval a rozdělil CSS styly. Kdybych měl více času, naprogramoval bych administrátorské rozhraní pro správu produktů a možnost řazení zboží podle ceny.

Projekt splnil zadané cíle 2. fáze — statický frontend byl převeden na plnohodnotný e-shop s persistencí, košíkem, checkoutem a uložením objednávek. Architektura Repository + DTO umožňuje snadné rozšiřování (např. přidání administrace produktů, uživatelských účtů, nebo migrace na MySQL pouhou změnou DSN v Database.php).